

Recherche sémantique & RAG par embeddings

Comment le tuteur trouve le bon contenu par le sens — expliqué depuis ton code

Plateforme d'apprentissage adaptatif · Yassine Ben Nessib · EspritTech — expliqué à partir du vrai code source

En une phrase — On transforme le texte en **vecteurs qui capturent le sens** (embeddings), puis on compare les sens avec la **similarité cosinus** pour récupérer le contenu de cours le plus proche pour le tuteur IA. Le cœur est `embedding_service.py`, utilisé par `rag_service.py`.

1 L'idée en une minute

L'ancienne recherche liait une question à un concept en comptant les **mots communs**. Elle échoue sur les paraphrases : « comment trouver où une courbe coupe zéro ? » n'a aucun mot commun avec « méthode de la bisection », alors qu'ils veulent dire la même chose. On est donc passé à la recherche **sémantique** : comparer les **sens**, pas les mots.

2 Où ça vit dans le code

`app/services/embedding_service.py`

Transforme le texte en vecteurs et compare les sens.

`app/services/rag_service.py`

Utilise les embeddings pour récupérer du contenu pour le tuteur.

`app/services/graph_service.py`

Le graphe de connaissances Neo4j (concepts & liens).

`sentence-transformers`

Le modèle pré-entraîné qui produit les vecteurs.

3 Charger le modèle en toute sécurité

EMBEDDING_SERVICE.PY — IMPORT SÉCURISÉ (DÉGRADATION GRACIEUSE)

```
try:
    import numpy as np
    from sentence_transformers import SentenceTransformer
    EMBEDDINGS_AVAILABLE = True
except Exception as exc:
    EMBEDDINGS_AVAILABLE = False
    logger.info("Embeddings disabled (sentence-transformers not installed)")

_MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
```

Ce que ça fait, étape par étape :

- Le `try / except` signifie : essayer d'importer la librairie IA ; si elle manque, ne pas planter — on met un drapeau `EMBEDDINGS_AVAILABLE = False` et la plateforme retombe sur la recherche par mots-clés.
- `_MODEL_NAME` est le modèle choisi : petit, rapide et **multilingue** (français + anglais) — environ 120 Mo, tourne sur CPU.

4 Transformer un texte en vecteur

EMBEDDING_SERVICE.PY — ENCODE()

```
def encode(text):
    model = _get_model()                # load the model on first use (lazy)
    if model is None or not text:
        return None
    if text in _cache:                   # already computed? reuse it
        return _cache[text]
    vec = model.encode(text, normalize_embeddings=True)
    _cache[text] = vec
    return vec
```

Ce que ça fait, étape par étape :

- `_get_model()` charge le modèle **une seule fois**, à la première utilisation (chargement paresseux), pour garder un démarrage rapide.
- `model.encode(text, normalize_embeddings=True)` est la ligne clé : elle renvoie l'**embedding** — une liste de nombres décrivant le sens. `normalize` met tous les vecteurs à la même longueur, ce qui simplifie le calcul de similarité.
- `_cache` est un dict Python qui **retient** les vecteurs déjà calculés, pour ne jamais encoder deux fois le même texte (plus rapide).

5 Comparer deux sens

EMBEDDING_SERVICE.PY — COSINE() + BEST_MATCH()

```
def cosine(a, b):  
    return float(np.dot(a, b))          # for normalized vectors, dot = cosine  
  
def best_match(query, candidates):      # candidates = list of (id, text)  
    q_vec = encode(query)  
    best_id, best_sim = None, -1.0  
    for cid, text in candidates:  
        v = encode(text)  
        sim = cosine(q_vec, v)  
        if sim > best_sim:              # keep the closest meaning  
            best_sim, best_id = sim, cid  
    return best_id, best_sim
```

Ce que ça fait, étape par étape :

- `cosine(a,b)` mesure à quel point deux vecteurs sont alignés. Comme les vecteurs sont normalisés, le cosinus est simplement leur **produit scalaire** (`np.dot`). Proche de 1 = même sens ; proche de 0 = sans rapport.
- `best_match` encode la `query` de l'étudiant, puis **parcourt** tous les textes candidats, calcule la similarité, et garde celui au score le plus haut.
- Elle renvoie l'identifiant du meilleur contenu et sa similarité — exactement ce dont le RAG a besoin pour récupérer.

SIMILARITÉ COSINUS (1 = SENS IDENTIQUE, 0 = SANS RAPPORT)

$\text{cosine}(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|)$ → avec des vecteurs normalisés, c'est juste $a \cdot b$

6 Le pipeline RAG complet à l'exécution

- 1 **Encoder le contenu** — chaque texte de leçon/concept est transformé en vecteur (une fois).
- 2 **Encoder la question** — la question de l'étudiant devient un vecteur de la même façon.
- 3 **Trouver le plus proche** — `best_match()` compare les sens et choisit le meilleur contenu.
- 4 **Augmenter le prompt** — le contenu récupéré est placé dans le prompt du tuteur comme contexte.
- 5 **Générer & vérifier** — le LLM répond avec ce contexte ; les maths sont ensuite vérifiées par SymPy.

C'est le sens de **RAG = Retrieval-Augmented Generation** : d'abord *récupérer* les bons faits, puis *générer* la réponse à partir d'eux. Notre version est du **GraphRAG** car le graphe de connaissances Neo4j guide aussi quels concepts récupérer.

7 Forces & limites

Forces

- Trouve le contenu par le sens, même entre langues
- Ancre le tuteur → moins de réponses inventées
- Rapide (un simple produit scalaire)
- Le cache évite de recalculer

Limites

- La qualité dépend du modèle d'embeddings
- La façon de découper le contenu compte beaucoup
- Mauvaise récupération → mauvaise réponse
- Nécessite un bon contenu écrit

Note honnête — Si `sentence-transformers` n'est pas installé sur le serveur, le drapeau est `False` et la plateforme retombe automatiquement sur l'ancien appariement par mots-clés — elle ne plante jamais. Installation : `pip install sentence-transformers`.

8 Q&R

QUESTIONS DE SOUTENANCE / ENTRETIEN (DONT « CE BLOC FAIT QUOI ? »)

Q Que renvoie `encode()` ?

R L'embedding d'un texte : un vecteur de nombres normalisé qui représente son sens. Des textes proches donnent des vecteurs proches.

Q Pourquoi `cosine()` n'est qu'un produit scalaire ici ?

R Parce qu'on normalise les vecteurs à l'encodage. Pour des vecteurs de longueur 1, le cosinus de l'angle égale leur produit scalaire, calculé vite avec `np.dot`.

Q Que fait `best_match()`, ligne par ligne ?

R Elle encode la question, parcourt chaque texte candidat, calcule la similarité cosinus, et renvoie le candidat au meilleur score.

Q Qu'est-ce que le RAG et pourquoi l'utiliser ?

R Retrieval-Augmented Generation : récupérer le contenu pertinent d'abord, puis laisser le modèle répondre avec. Cela ancre les réponses dans notre contenu et réduit les hallucinations.

Q Que se passe-t-il si la librairie manque ?

R Le `try/except` met `EMBEDDINGS_AVAILABLE = False` et on retombe sur la recherche par mots-clés. Dégradation gracieuse — rien ne plante.

Q Qu'est-ce que le GraphRAG ?

R Un RAG où un graphe de connaissances (Neo4j, avec des liens COVERS/REQUIRES/REMIEDIATES_TO) guide la récupération, pas seulement la similarité brute des vecteurs.

9 Termes clés

Embedding — Un vecteur de nombres capturant le sens d'un texte.

Normaliser — Mettre chaque vecteur à la même longueur (longueur 1).

Similarité cosinus — Proximité de sens = produit scalaire de vecteurs normalisés.

Cache — Mémoriser les résultats pour ne pas recalculer.

Chargement paresseux — Charger le modèle seulement à la première utilisation.

RAG — Récupérer le contenu pertinent, puis générer la réponse.

Hallucination — Quand un modèle invente une réponse fausse.

Dégradation gracieuse — Continuer à fonctionner (repli) quand une partie manque.